

Grafos I

24/11/2025

Ficheiro ZIP

- Está disponível no Moodle um ficheiro ZIP de suporte aos tópicos de hoje
- Ficheiros de texto que armazenam diferentes tipos de grafos
- O tipo abstrato Grafo usando o TAD SortedList
- Versão “simples”, que permite trabalho autónomo de desenvolvimento e teste

Sumário

- Grafos: Terminologia; Exemplos de aplicação; Propriedades
- O Tipo de Dados **Grafo**
- Possíveis estruturas de dados
- Operações básicas
- Desempenho computacional
- Exercícios / Tarefas 
- Sugestão de leitura

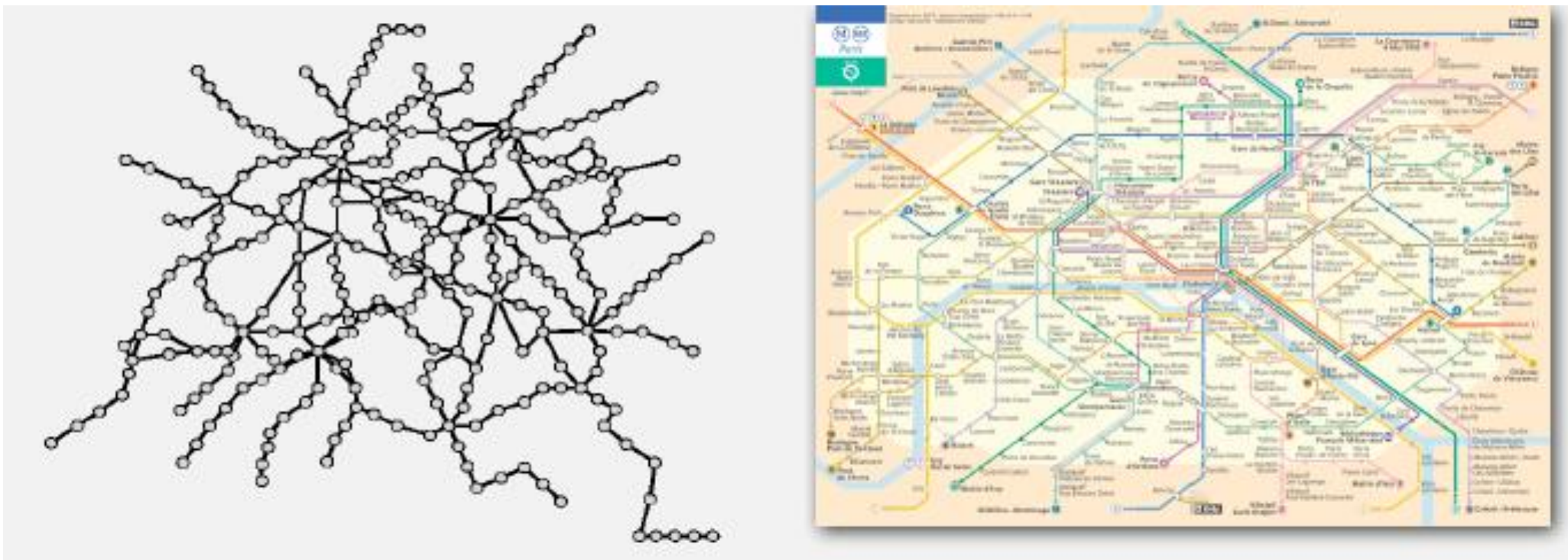
Grafos

– Motivação + Exemplos

Grafos – Porquê estudar ?

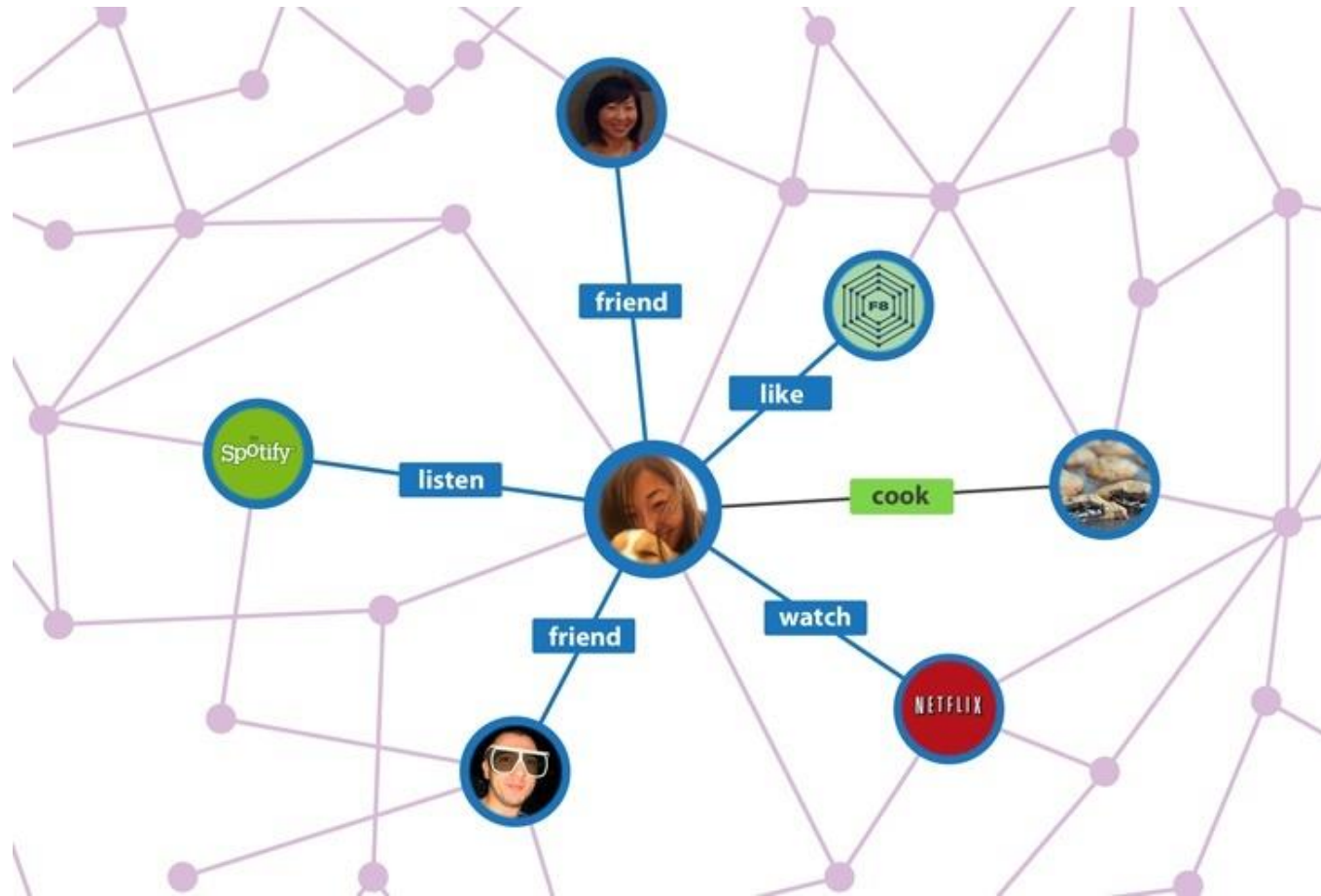
- Abstração útil, que permite modelar/representar **entidades** e as suas **relações**
- Teoria dos Grafos: subárea das Ciências da Computação e da Matemática Discreta
 - **Problemas / Algoritmos / Aplicações**
- Centenas de algoritmos
- Milhares de aplicações práticas

Aplicação – Redes de transportes



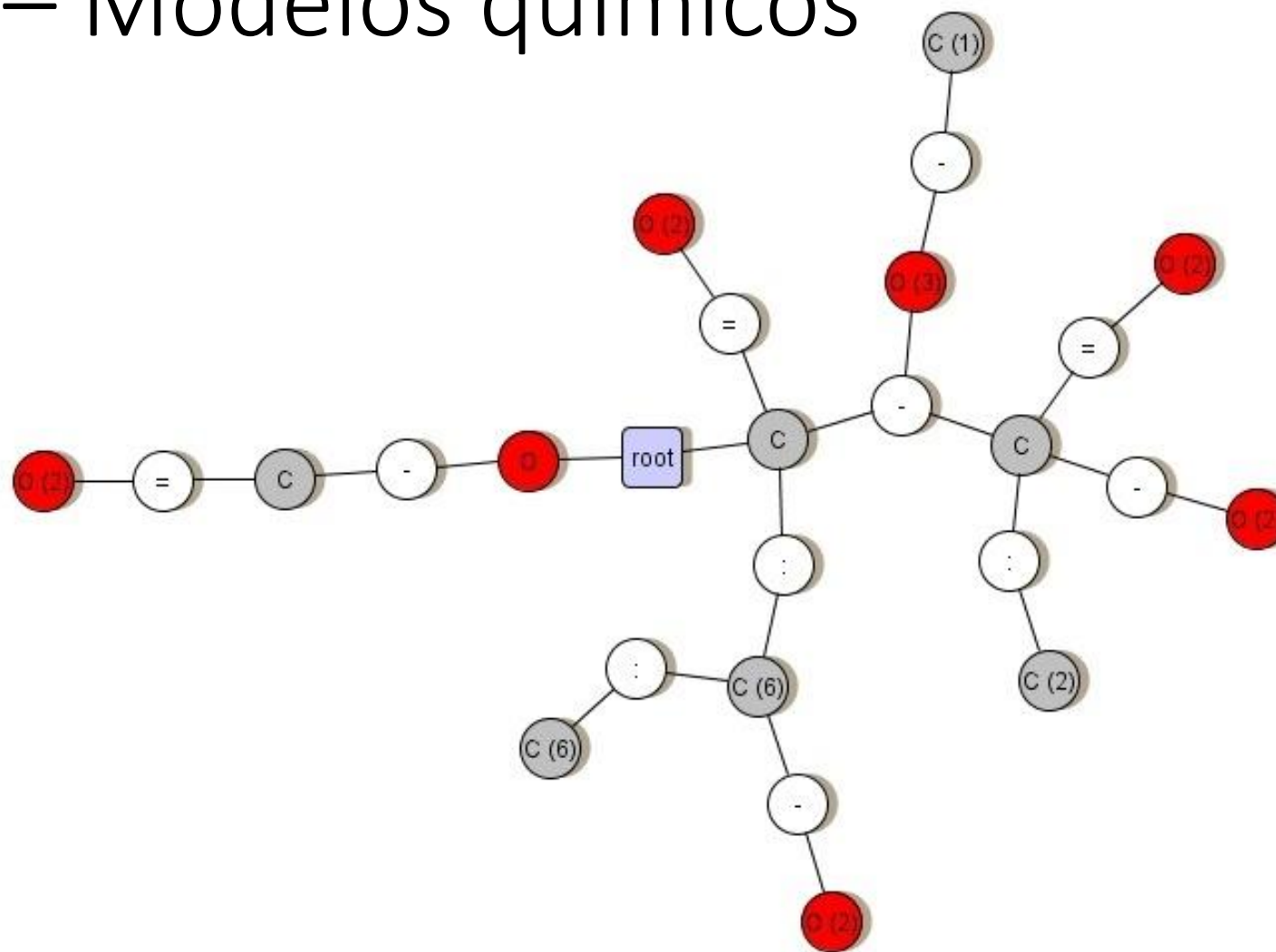
[Sedgewick & Wayne]

Aplicação – Redes sociais



[Quora]

Aplicação – Modelos químicos



[Quora]

Grafos – Aplicações

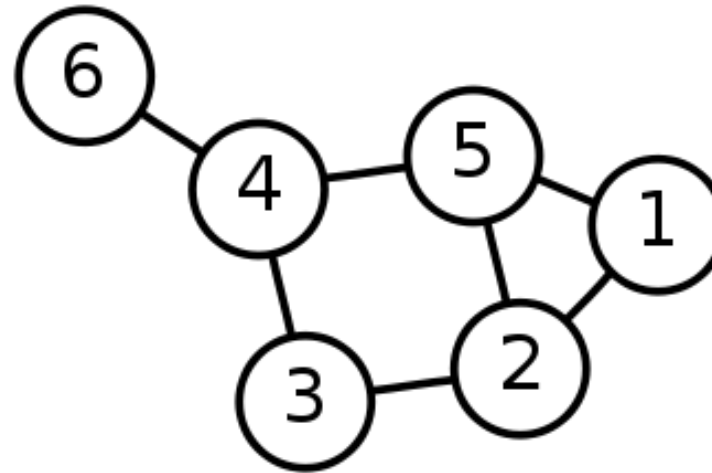
Graph	Vertex	Edge
communication	telephone, computer	cable
circuit	gate, register, processor	wire
mechanical	joint	rod, beam, spring
financial	stock, currency	transaction
transportation	street intersection, airport	highway, airway route
Internet	class C network	connection
game	board position	legal move
relationship	person	friendship
neural network	neuron	synapse
protein network	protein	protein-protein interaction
chemical compound	molecule	bond

[Sedgewick & Wayne]

Grafos

– Terminologia + Propriedades

Grafos – Definição



[Wikipedia]

- $G(V, E)$
- Quando muito uma **aresta** ligando qualquer **par de vértices distintos**
- $e_i = (v_j, v_k)$
 - v_j e v_k são **vértices adjacentes**
 - e_i é **incidente** em v_j e em v_k

Grafos – Propriedades

- Número máximo de arestas = $V \times (V - 1) / 2$
 - Grafo completo : K_V
- Grau de um vértice
 - Número de arestas incidentes nesse vértice
 - Grau máximo ?
- Grafo regular
 - Todos os vértices têm o mesmo grau k : grafo *k -regular*

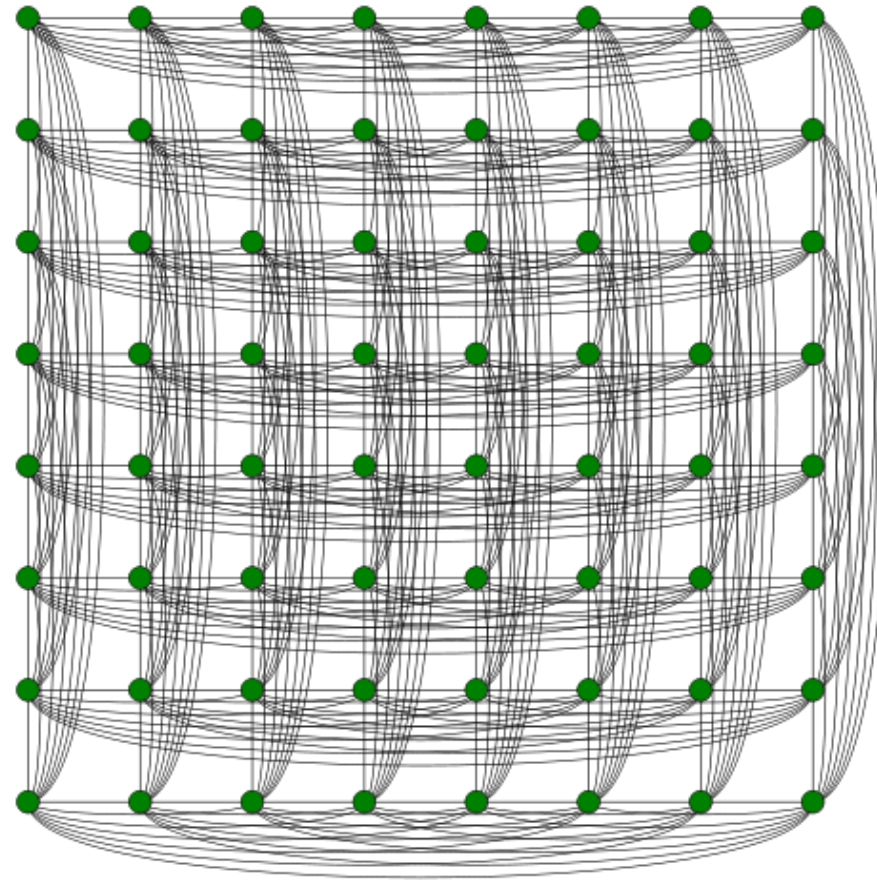
Grafo dos movimentos da torre no xadrez

64 vértices

448 arestas

14-regular

número cromático 8



[Wikipedia]

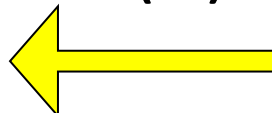
Grafos – Propriedades

$$\sum grau(v) = 2 \times E$$

- Grau médio – Average vertex degree

$$AVD = (2 \times E) / V$$

- Grafo **denso** : E em $\Theta(V^2)$ e AVD em $\Theta(V)$

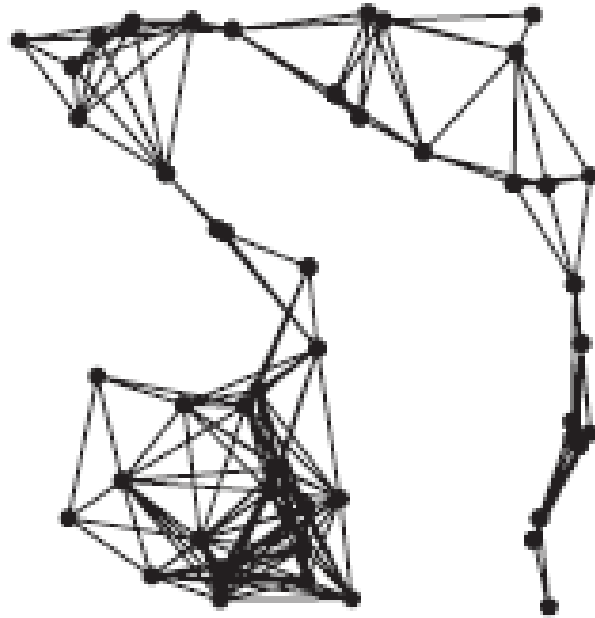
- Grafos densos vs. grafos **esparsos** 

- Densidade

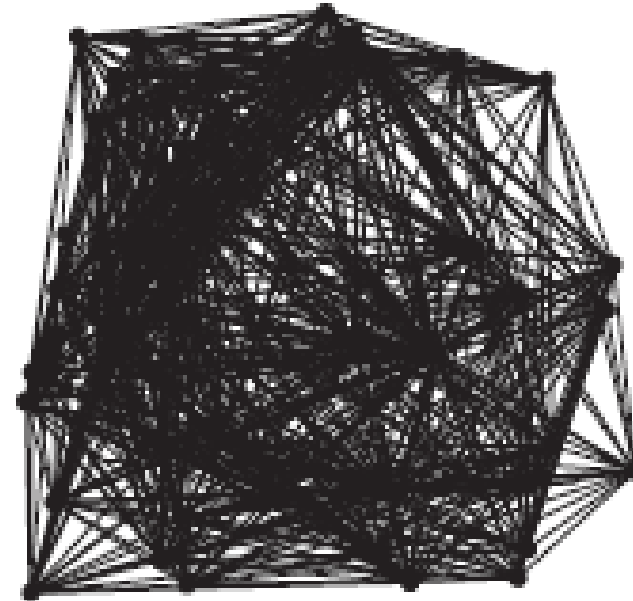
$$D = (2 \times E) / (V \times (V - 1))$$

Grafo **esparso** vs Grafo **denso**

sparse ($E = 200$)



dense ($E = 1000$)



Two graphs ($V = 50$)

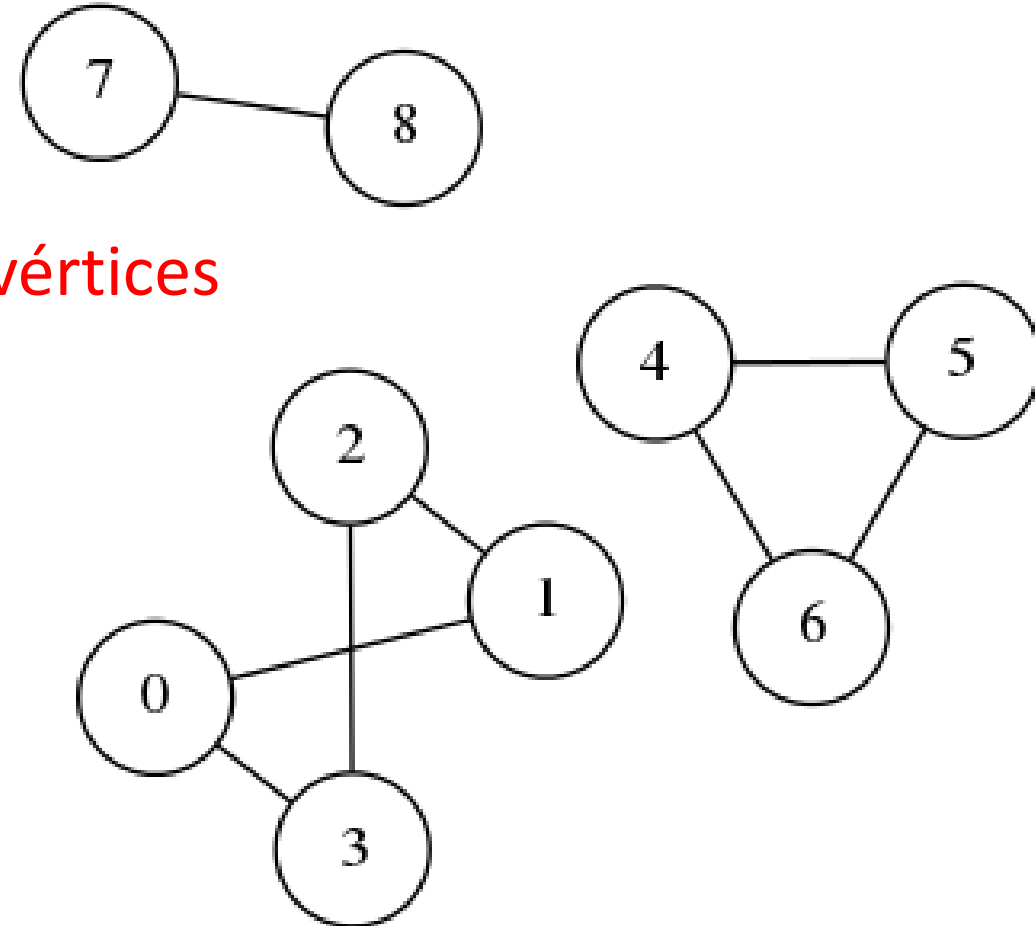
[Sedgewick & Wayne]

Grafos – Passeio / Trajeto / Caminho

- Um **passeio** é uma qualquer sequência de vértices adjacentes
 - **Comprimento** do passeio: **nº de arestas** que o constituem
- Um **trajeto** é um passeio constituído por **arestas distintas**
 - Um **circuito** é um trajeto de comprimento não nulo, que começa e acaba no mesmo vértice
- Um **caminho** é um passeio constituído por **arestas e vertices distintos**
 - Um **ciclo** é um caminho de comprimento não nulo, que começa e acaba no mesmo vértice

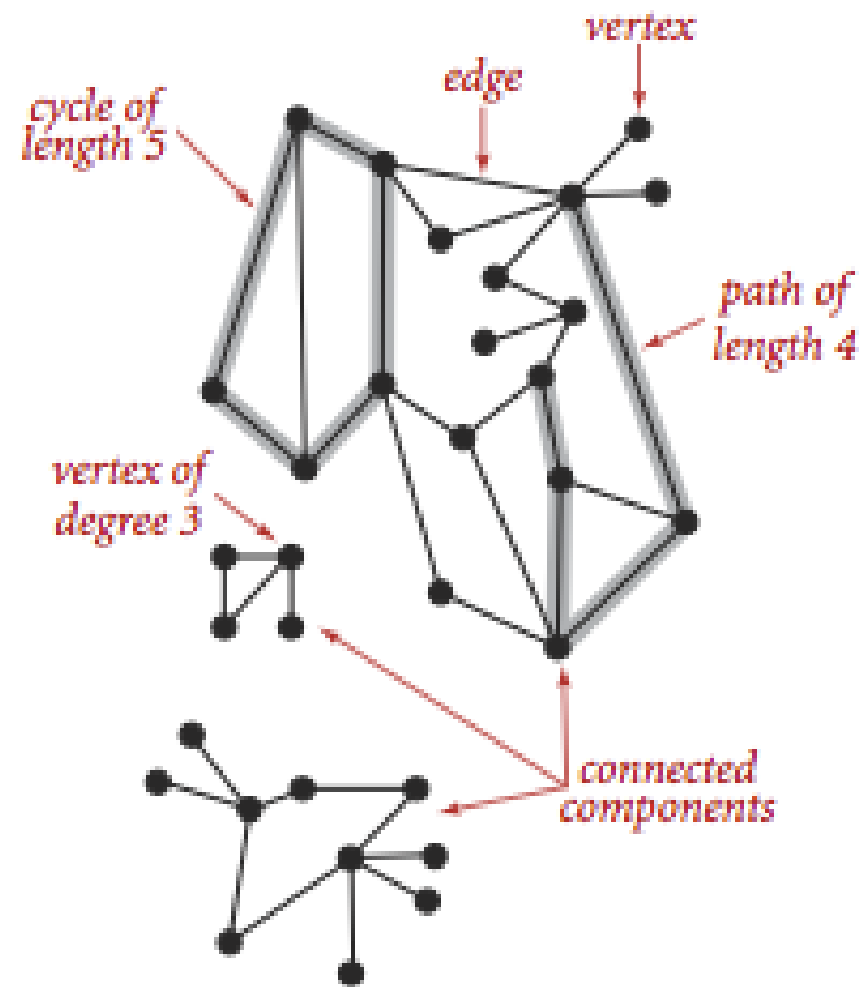
Grafos – Conectividade

- Grafo **conexo**
 - Existe um **caminho** entre cada **par de vértices**
 - Um único **componente conexo**



[martinbroadhurst.com/]

Grafos – Resumo



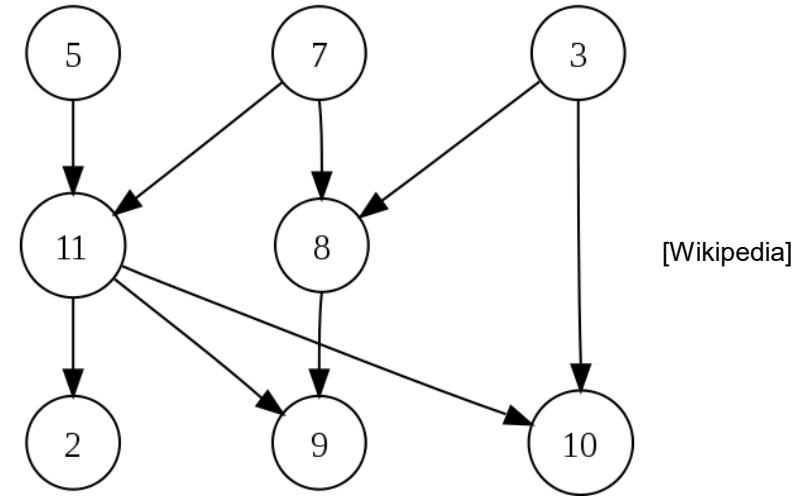
Anatomy of a graph

[Sedgewick & Wayne]

Grafos Orientados

Grafos Orientados

- $G(V,E)$
- Grafo **orientado**
 - As **arestas orientadas** definem uma adjacência unidirecional
- $e_i = (v_j, v_k)$
 - v_j é o vértice **origem** e v_k o vértice **destino**
 - v_k é **adjacente** a v_j
 - e_i é **incidente** em v_k



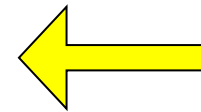
Grafos Orientados – Aplicações

Digraph	Vertex	Directed Edge
transportation	street intersection	one-way street
web	web page	hyperlink
food web	species	predator-prey relationship
scheduling	task	precedence constraint
financial	bank	transaction
cell phone	person	placed call
infectious disease	person	infection
game	board position	legal move
citation	journal article	citation
object class	object	pointer
inheritance hierarchy	class	inherits from
control flow	code block	jump

[Sedgewick/Wayne]

Grafos Orientados – Propriedades

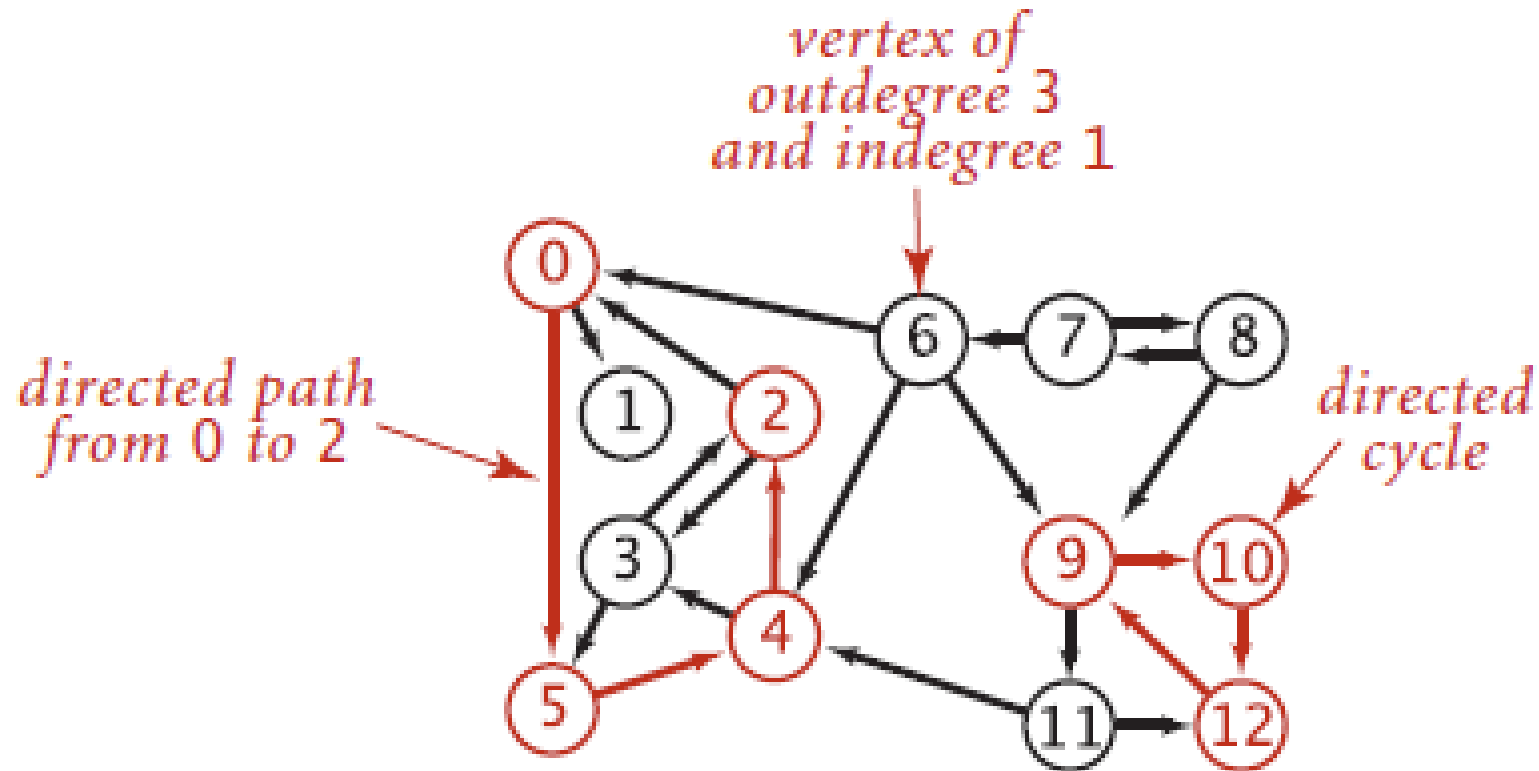
- Nº máximo de arestas = $V \times (V - 1)$
 - Grafo orientado completo
- In-Degree e Out-Degree associado a cada vértice
 - Vértice fonte (“source”) ?
 - Vértice sumidouro (“sink”) ?
- Densidade de um grafo orientado
 - Grafos orientados densos vs. esparsos



Grafos Orientados – Passeio / Caminho

- Um **passeio orientado** é uma sequência de vértices
 - Cada vértice (exceto o primeiro) é **adjacente** ao seu **predecessor**
- **Caminho orientado** : arestas e vértices distintos
- **Ciclo orientado** : caminho orientado com o mesmo vértice inicial e final
- Vértice t é **alcançável** a partir do vértice s ?
 - Existe um **caminho orientado** de s para t

Grafos Orientados – Resumo

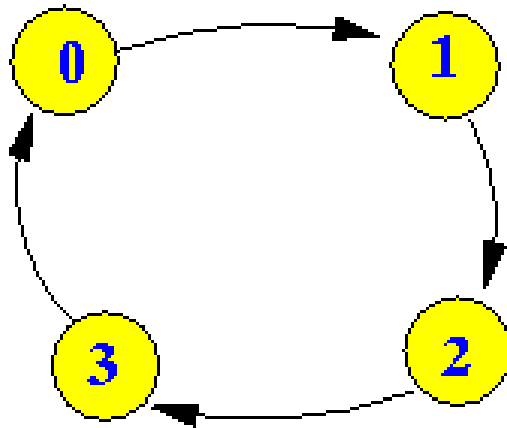


[Sedgewick/Wayne]

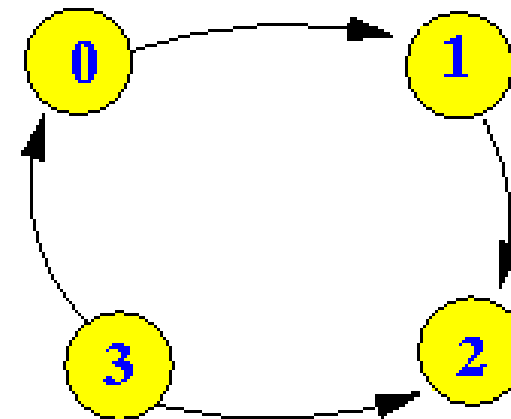
Grafos Orientados – Conectividade

- Grafo orientado fracamente conexo
 - Substituir as arestas orientadas por **arestas não-orientadas**
 - O grafo resultante é **conexo**
- Grafo orientado fortemente conexo
 - Existe um caminho entre cada par de vértices
 - Vértices **mutuamente alcançáveis** !!
 - **Um único componente** fortemente conexo

Exemplo – Conectividade



Strongly connected

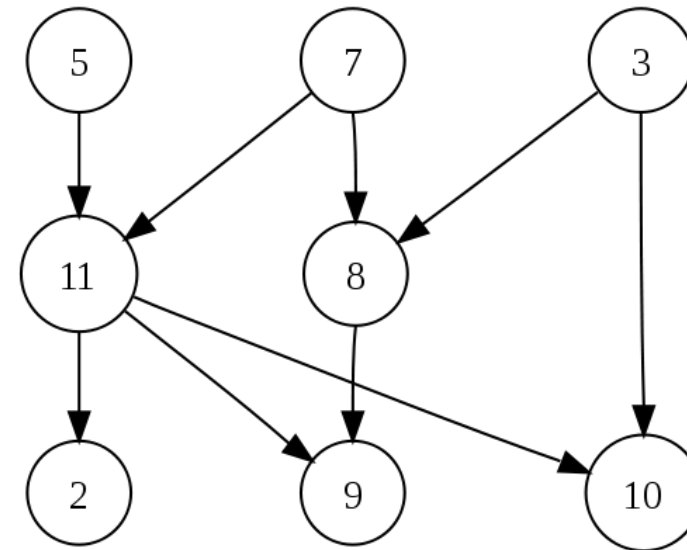


Not strongly connected

[cs.emory.edu]

Grafos Orientados **Acíclicos**

- Directed Acyclic Graphs (**DAGs**)
- Um grafo orientado que **não** contém **qualquer ciclo** !
 - Relações de **precedência**

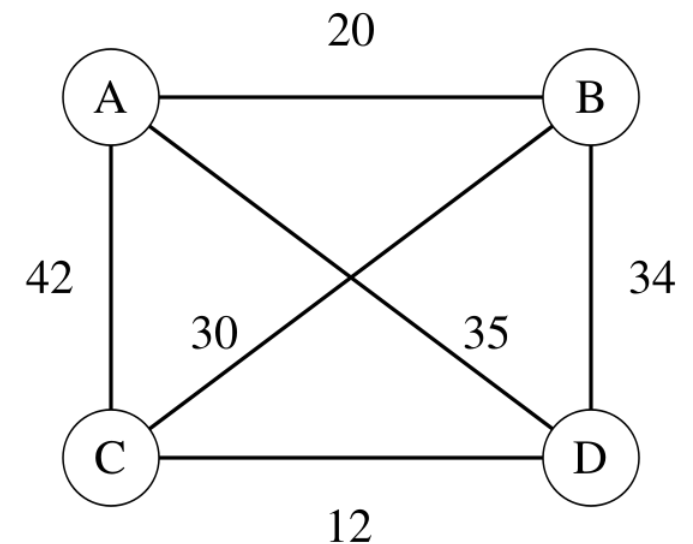


[Wikipedia]

Redes

Redes – Definição

- Uma rede é um grafo / grafo orientado com “**pesos**” associados às suas arestas
 - **Weighted graph / digraph**
 - Associar **um ou mais valores** a cada aresta
 - **Custo, distância, capacidade, ...**

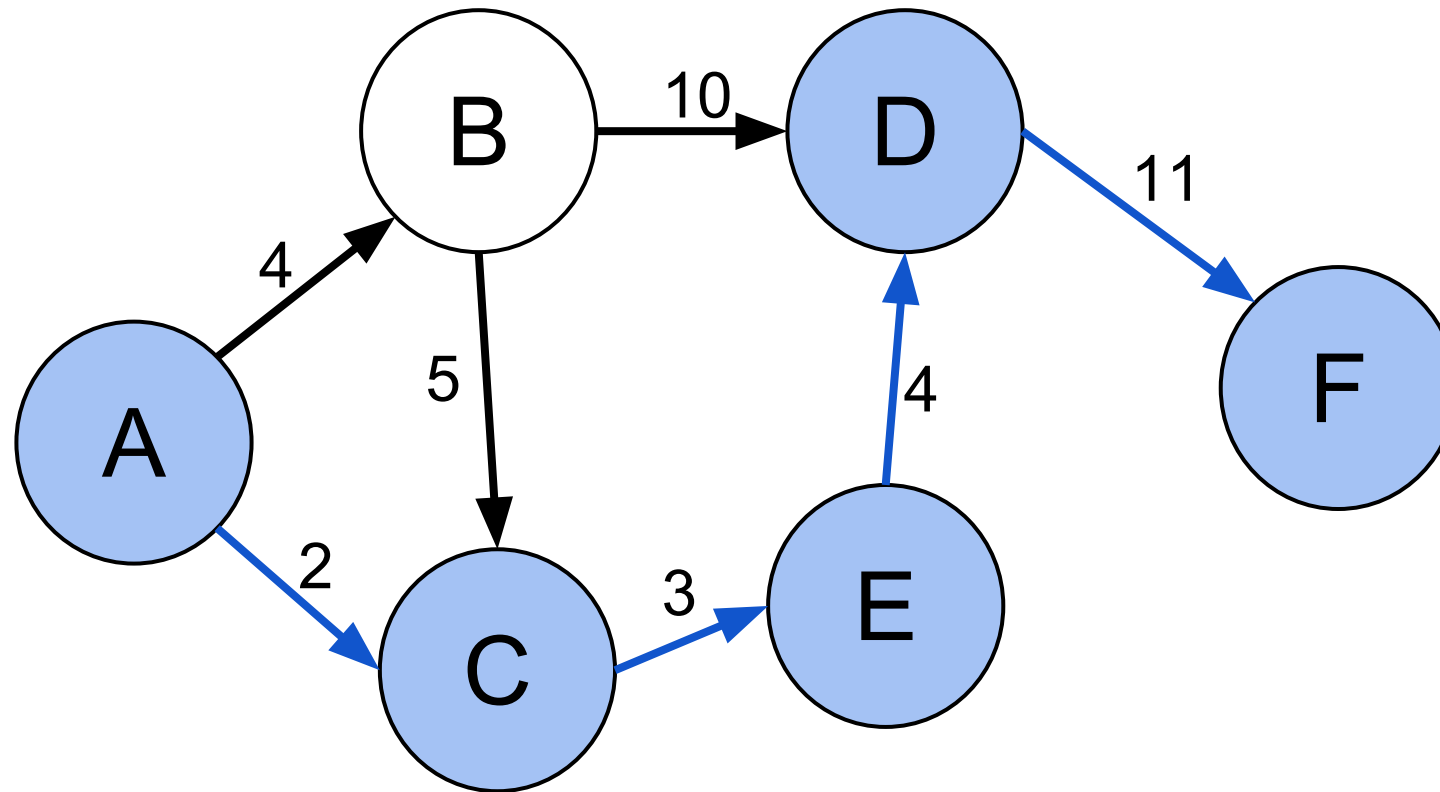


[Wikipedia]

Redes – Caminho / Caminho mais curto

- Existe um **caminho** entre os vértices **s** e **t** ?
- Qual é o **caminho mais curto** entre **s** e **t** ?
 - **Soma** das **distâncias** associadas a cada **aresta**

Caminho mais curto entre A e F – Custo ?



[Wikipedia]

TAD GRAFO

- Que funcionalidades?
- Como representar?

TAD Grafo

```
Graph* GraphCreate(unsigned int V);           // Apenas com V vértices
GraphDestroy(Graph** g);
Graph* GraphCopy(Graph* g);
Graph* GraphFromFile(FILE* f);
unsigned int GraphGetNumVertices(Graph* g);
unsigned int GraphGetNumEdges(Graph* g);
...
```

TAD Grafo

...

```
int GraphGetVertexDegree(Graph* g, unsigned int v);
```

```
int GraphGetDegree(Graph* g);
```

```
double GraphGetAverageDegree(Graph* g);
```

...

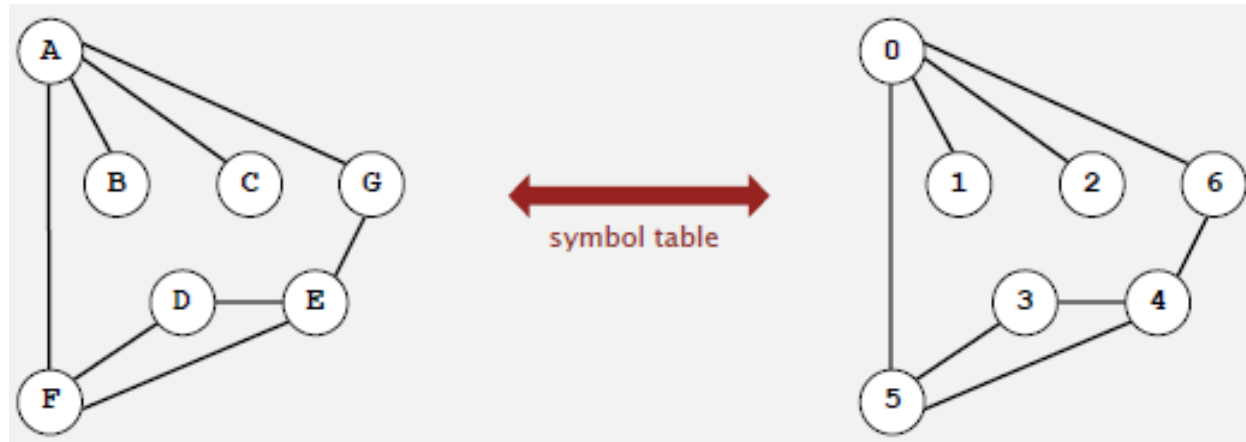
```
int GraphAddEdge(Graph* g, unsigned int v, unsigned int w);
```

```
List* GraphGetAdjacentTo(Graph* g, unsigned int v);
```

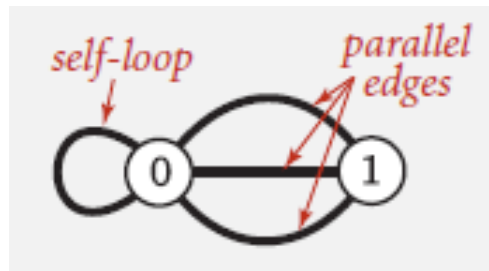
...

TAD Grafo – Vértices

- Identificados por um valor inteiro de **0** a **$V - 1$**
- Usar dicionários para mapear esses IDs noutros identificadores

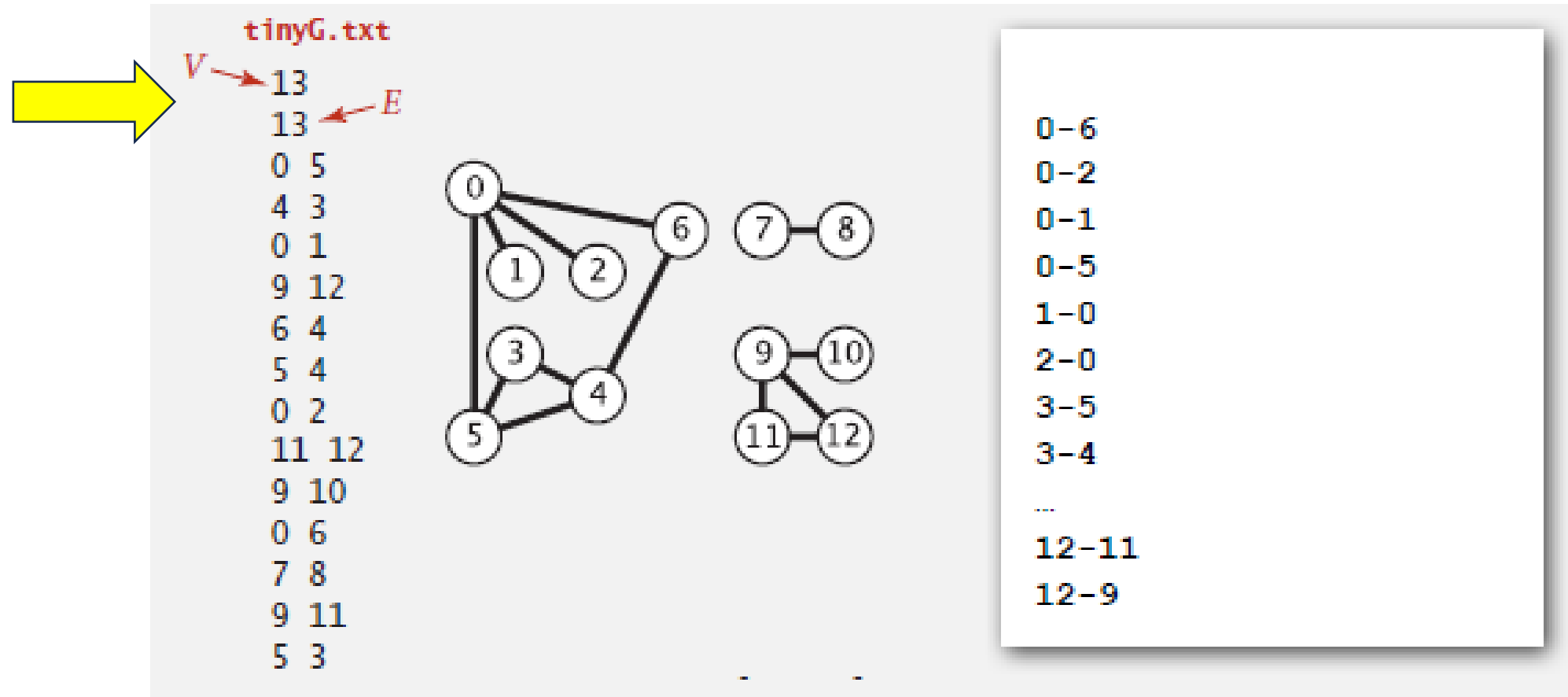


- **Não** são permitidos **lacetes** nem **arestas paralelas** !!



[Sedgewick/Wayne]

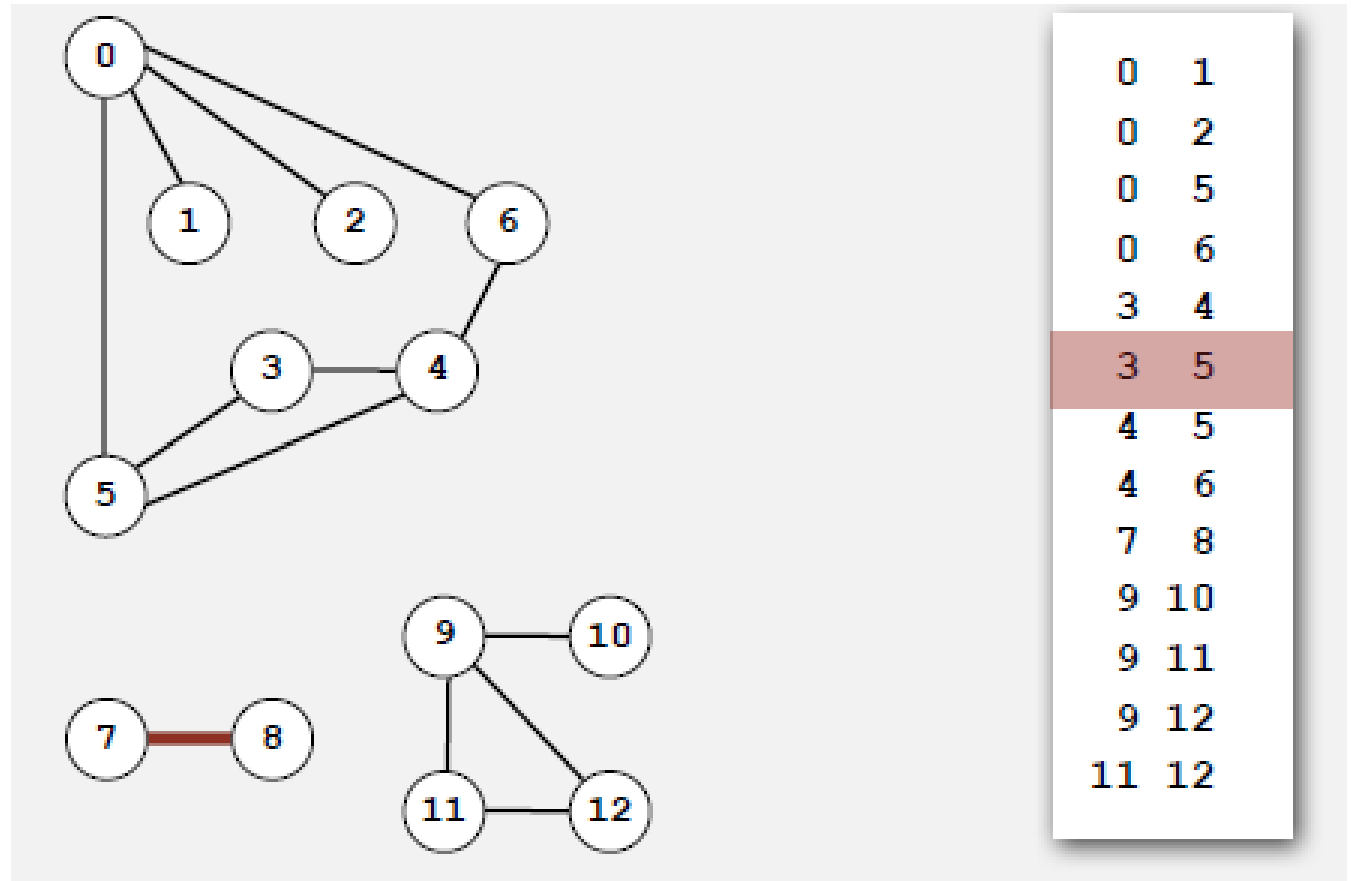
Representação em ficheiro – Lista de arestas



[Sedgewick/Wayne]

Representação – Lista ordenada de arestas

- Armazenar usando uma **lista ligada de arestas**

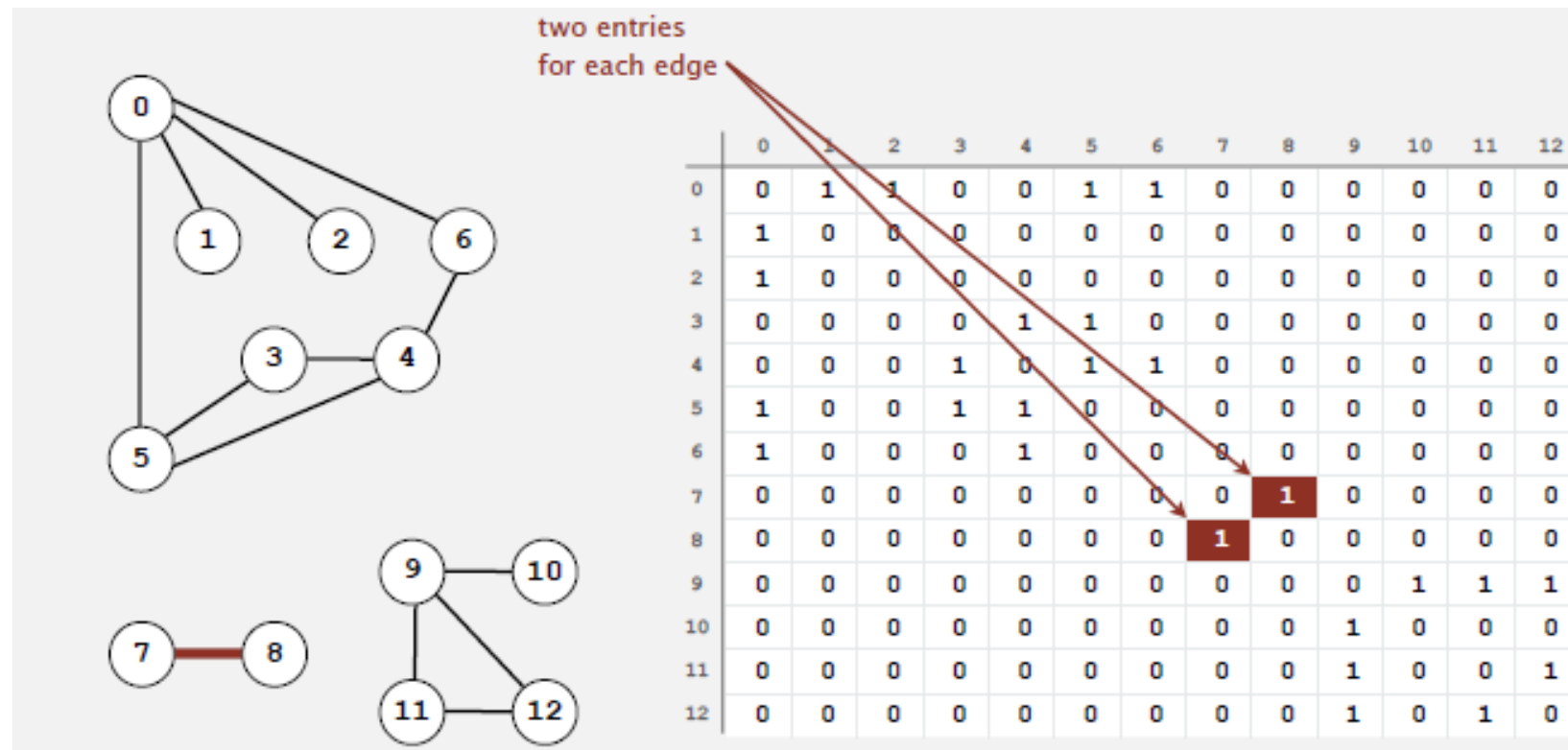


[Sedgewick/Wayne]

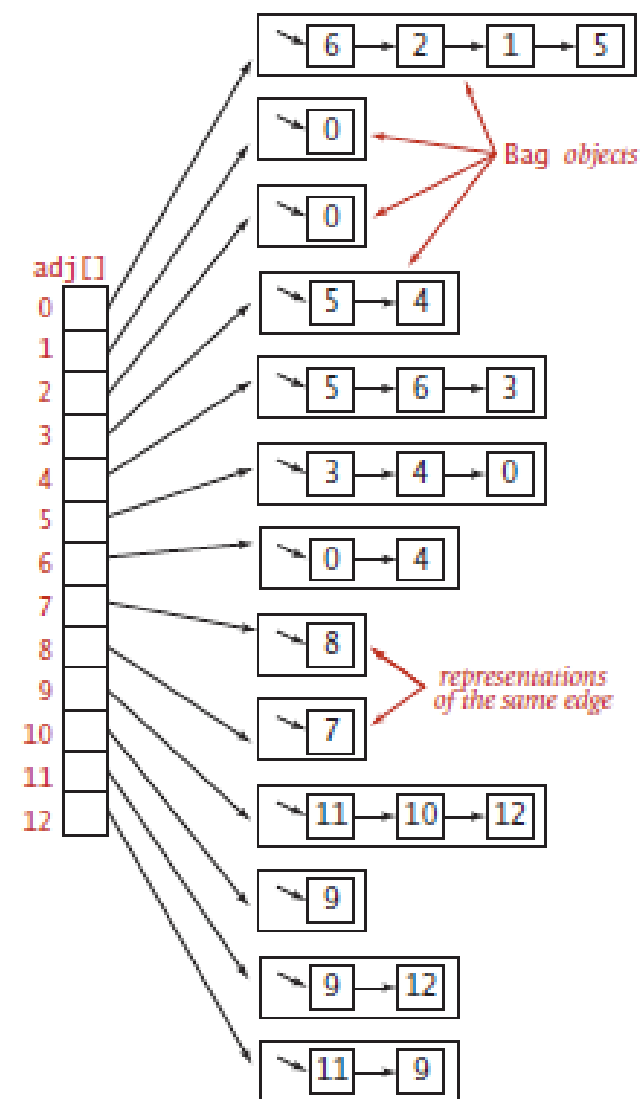
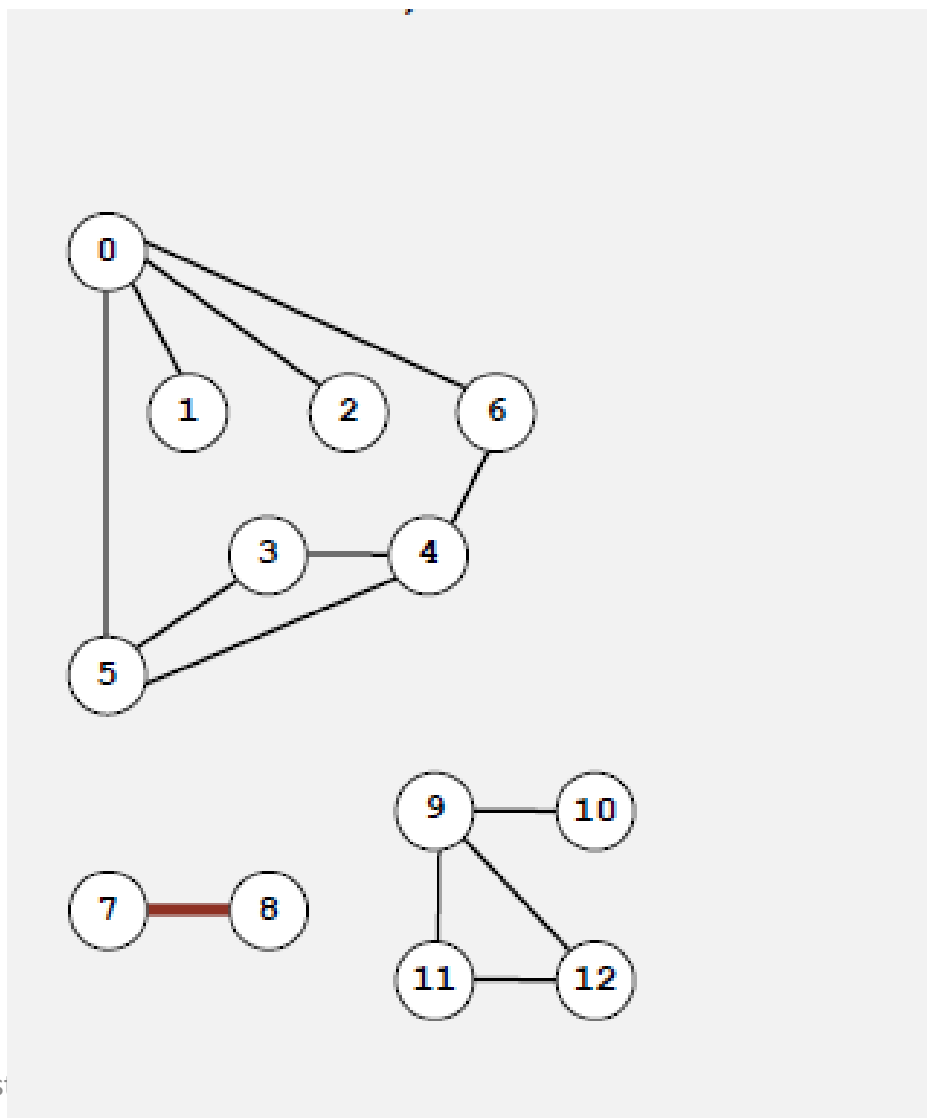
Representação – Matriz de adjacência

- Array de V^2 booleanos
- Cada **aresta** é representada **duas vezes**
 - Porquê ?

[Sedgewick/Wayne]



Representação – Listas de adjacências



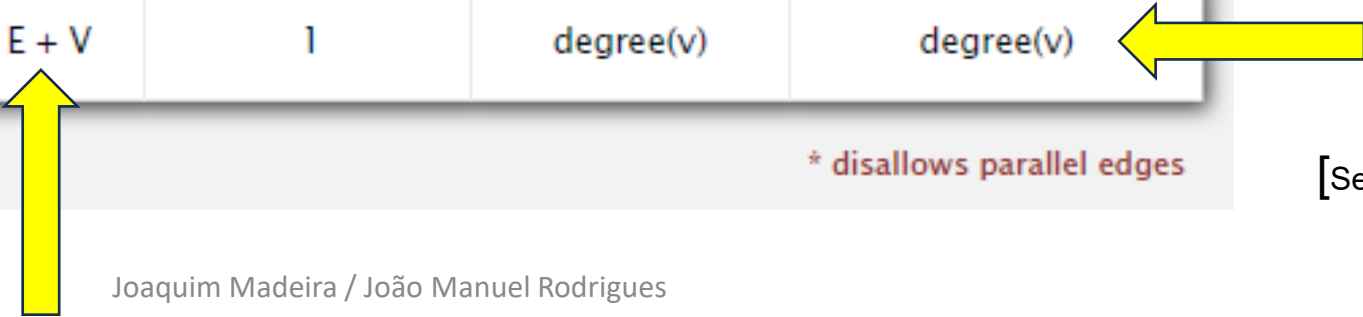
[Sedgewick/Wayne]

Representação – Eficiência computacional

- Na prática: usar a representação em **listas de adjacências**
- Os **grafos** do mundo real são habitualmente **esparsos** !!
- Algoritmos iteram sobre os **vértices adjacentes** a um vértice dado

representation	space	add edge	edge between v and w?	iterate over vertices adjacent to v?
list of edges	E	1	E	E
adjacency matrix	V^2	1 *	1	V
adjacency lists	$E + V$	1	degree(v)	degree(v)

* disallows parallel edges



[Sedgewick/Wayne]

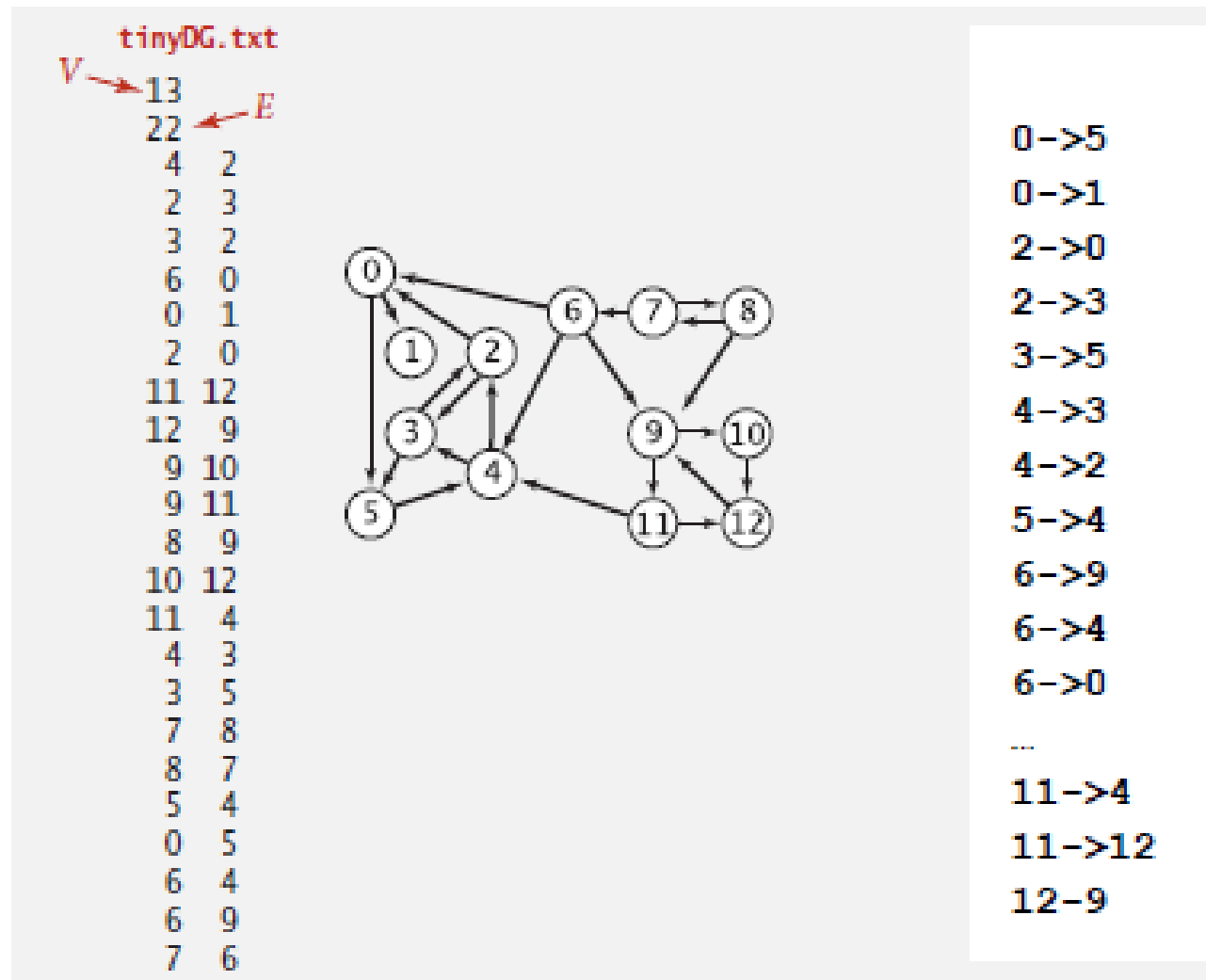
TAD GRAFO ORIENTADO

- Que funcionalidades?
- Como representar?

TAD Grafo Orientado

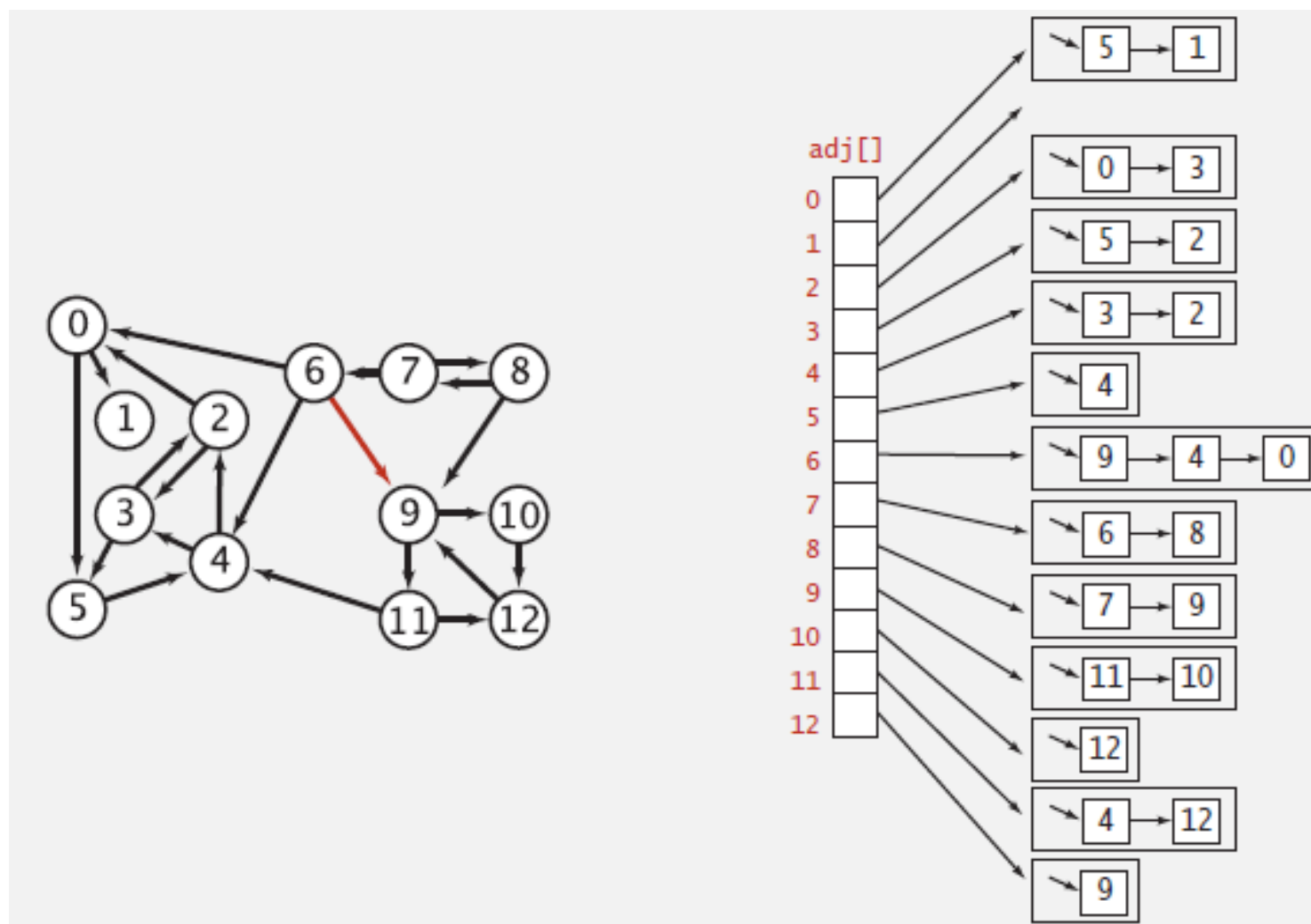
```
Digraph* DigraphCreate(unsigned int V);    // Apenas com V vértices
DigraphDestroy(Digraph** g);
Digraph* DigraphCopy(Digraph* g);
Digraph* DigraphCreateReverse(Digraph* g);
Digraph* DigraphFromFile(FILE* f);
unsigned int DigraphGetNumVertices(Digraph* g);
unsigned int DigraphGetNumEdges(Digraph* g);
...
```

Representação em ficheiro – Lista de arestas



[Sedgewick/Wayne]

Representação – Listas de adjacências



[Sedgewick/Wayne]

Representação – Eficiência computacional

- Na prática: usar a representação em **listas de adjacências**
- Os **grafos orientados** do mundo real são habitualmente **esparsos** !!
- Algoritmos iteram sobre os **vértices adjacentes** a um vértice dado

representation	space	insert edge from v to w	edge from v to w?	iterate over vertices adjacent from v?
list of edges	E	1	E	E
adjacency matrix	V^2	1†	1	V
adjacency list	E + V	1	outdegree(v)	outdegree(v)

† disallows parallel edges

[Sedgewick/Wayne]

O TAD GRAFO

TAD GRAFO – Decisão – Um só TAD !!

- Representar **Grafos / Grafos Orientados / Redes** 
- O que é **comum / diferente** ?
- Operações **básicas**, apenas !! 
- **Lista** ligada de **vértices** + **Listas** ligadas de **adjacências** 
- Usar o **TAD Sorted List** !!
- **Módulos adicionais** para os vários **algoritmos** !! 

TAD GRAFO – Questões – Como definir ?

- As operações básicas
- As operações auxiliares
- O cabeçalho da estrutura de dados
- Um nó da lista de vértices
- Um nó das listas de adjacências

TAD GRAFO – Cabeçalho / Vértice / Aresta



```
struct _GraphHeader {  
    unsigned short isDigraph;  
    unsigned short isComplete;  
    unsigned short isWeighted;  
    unsigned int numVertices;  
    unsigned int numEdges;  
    List* verticesList;  
};
```



```
struct _Vertex {  
    unsigned int id;  
    unsigned int inDegree;  
    unsigned int outDegree;  
    List* edgesList;  
};
```



```
struct _Edge {  
    unsigned int adjVertex;  
    int weight;  
};
```

- Os **atributos do cabeçalho** permitem classificar o grafo
- Se o grafo for **não-orientado**, é suficiente armazenar o (**out**)**Degree** de cada vértice

TAD GRAFO – Criar e destruir grafos

```
typedef struct _GraphHeader Graph;

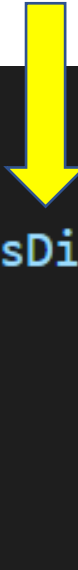
Graph* GraphCreate(unsigned short numVertices, unsigned short isDigraph,
                  unsigned short isWeighted);

Graph* GraphCreateComplete(unsigned short numVertices,
                          unsigned short isDigraph);

void GraphDestroy(Graph** p);

Graph* GraphCopy(const Graph* g);

Graph* GraphFromFile(FILE* f);
```



TAD GRAFO – Propriedades de um grafo

```
unsigned short GraphIsDigraph(const Graph* g);  
  
unsigned short GraphIsComplete(const Graph* g);  
  
unsigned short GraphIsWeighted(const Graph* g);  
  
unsigned int GraphGetNumVertices(const Graph* g);
```

TAD GRAFO – Propriedades de um grafo

- Distinguir entre grafo não-orientado e grafo orientado
- É responsabilidade do programador saber que funções pode ou não invocar !!

```
//  
// For a graph ←  
//  
double GraphGetAverageDegree(const Graph* g);  
  
//  
// For a graph ←  
//  
unsigned int GraphGetMaxDegree(const Graph* g);  
  
//  
// For a digraph ←  
//  
unsigned int GraphGetMaxOutDegree(const Graph* g);
```

TAD GRAFO – Propriedades de um **vértice**

- **Array** com os **IDs** dos **vértices adjacentes**
- **Array** com as **distâncias** aos **vértices adjacentes**

```
// Vertices
unsigned int* GraphGetAdjacentsTo(const Graph* g, unsigned int v);

// *** NEW ***
int* GraphGetDistancesToAdjacents(const Graph* g, unsigned int v);

//
// For a graph
//
unsigned int GraphGetVertexDegree(Graph* g, unsigned int v);

//
// For a digraph
//
unsigned int GraphGetVertexOutDegree(Graph* g, unsigned int v);
```

TAD GRAFO – Adicionar arestas

```

unsigned short GraphAddEdge(Graph* g, unsigned int v, unsigned int w);

unsigned short GraphAddWeightedEdge(Graph* g, unsigned int v, unsigned int w,
                                     int weight);
// CHECKING

unsigned short GraphCheckInvariants(const Graph* g);

// DISPLAYING on the console

void GraphDisplay(const Graph* g);

void GraphListAdjacents(const Graph* g, unsigned int v);

```

TAD GRAFO – Questões de implementação

- Como **atravessar** a lista de vértices ?
- Como **atravessar** uma lista de adjacências ?
- Usar o **iterador** do TAD Sorted List !! 
- Como **comparar vértices** ou **arestas** ?
- Como **adicionar** uma aresta ?
- Como devolver os índices dos **vértices adjacentes** ?
- ...

TAD GRAFO

- Funções de comparação para o TAD SORTED LIST
- Para a lista de vértices, comparar vértices usando os seus IDs
- Para a lista de arestas adjacentes, comparar arestas usando o seu vértice final

```
// The comparator for the VERTICES LIST
```

```
int graphVerticesComparator(const void* p1, const void* p2) {  
    unsigned int v1 = ((struct _Vertex*)p1)->id;  
    unsigned int v2 = ((struct _Vertex*)p2)->id;  
    int d = v1 - v2;  
    return (d > 0) - (d < 0);  
}
```

```
// The comparator for the EDGES LISTS
```

```
int graphEdgesComparator(const void* p1, const void* p2) {  
    unsigned int v1 = ((struct _Edge*)p1)->adjVertex;  
    unsigned int v2 = ((struct _Edge*)p2)->adjVertex;  
    int d = v1 - v2;  
    return (d > 0) - (d < 0);  
}
```


TAD GRAFO

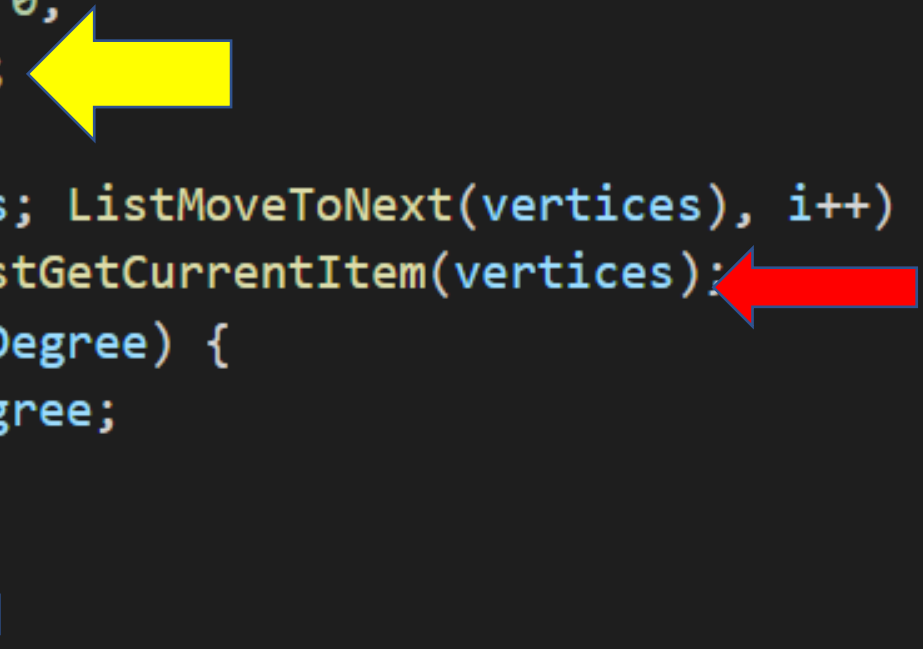
- Distinguir entre **grafo não-orientado** e **grafo orientado**
- É responsabilidade do **programador** saber que **funções** pode ou não **invocar** !!

```
//  
// For a graph  
//  
unsigned int GraphGetMaxDegree(const Graph* g) {  
    assert(g->isDigraph == 0);  
    return _GetMaxDegree(g);  
}  
  
//  
// For a digraph  
//  
unsigned int GraphGetMaxOutDegree(const Graph* g) {  
    assert(g->isDigraph == 1);  
    return _GetMaxDegree(g);  
}
```

TAD GRAFO – Grau máximo ?

- Função auxiliar !!
- Iterar sobre a lista de vértices
- Registrar o maior valor de **outDegree**

```
static unsigned int _GetMaxDegree(const Graph* g) {  
    List* vertices = g->verticesList;  
    if (ListIsEmpty(vertices)) return 0;  
  
    unsigned int maxDegree = 0;  
    ListMoveToHead(vertices);  
    int i = 0;  
    for (; i < g->numVertices; ListMoveToNext(vertices), i++) {  
        struct _Vertex* v = ListGetCurrentItem(vertices);  
        if (v->outDegree > maxDegree) {  
            maxDegree = v->outDegree;  
        }  
    }  
    return maxDegree;  
}
```





Exercícios / Tarefas

Exercício 1 – Verdadeiro ou Falso

Um **grafo não-orientado completo**, com n vértices, tem $(n^2 - n) / 2$ arestas.

Um **grafo orientado completo**, com n vértices, tem $(n^2 - n)$ arcos.

Um grafo orientado **fortemente conexo** pode conter um vértice isolado.

Se um grafo orientado é **fortemente conexo**, há sempre um caminho entre qualquer par de vértices.

Exercício 2 – Escolha múltipla

Considere um **grafo não-orientado** $G(V,E)$, representado usando a sua **matriz de adjacências**.

- a) Adicionar uma nova aresta ao grafo é uma operação de complexidade $O(V)$.
- b) Verificar se um nó é isolado é uma operação de complexidade $O(E)$.
- c) Criar uma cópia do grafo é uma operação de complexidade $O(V^2)$.
- d) Todas estão corretas.

Exercício 3 – Escolha múltipla

Considere um **grafo orientado** $G(V,E)$, representado usando a **lista ordenada dos seus nós** e, para cada nó, a sua **lista ordenada de adjacências**.

- a) No pior caso, verificar se um nó é isolado é uma operação de complexidade $O(E)$.
- b) No pior caso, adicionar uma nova aresta ao grafo orientado é uma operação de complexidade $O(V)$.
- c) Ambas estão corretas.
- d) Nenhuma está correta.

Tarefas – TAD GRAFO

- Analisar as funções desenvolvidas
- Completar o que falta !!
- Melhorar algumas das funções !!

Sugestão de Leitura

Sugestão de Leitura

- R. Sedgewick and K. Wayne, “*Algorithms*”, 4th. Ed., Addison-Wesley, 2011
 - Chapter 4